

# Modular ROS 2 Architecture for Visual-Based Navigation and Traffic Sign Recognition in a Differential Robot

Angelica N. Velazquez, Ximena L. Trejo, Brenda Cruz, Harum Kim, Cesar .M. Sánchez, Alondra Caspeta, Valeria Meneses.

1

**Abstract**—This project presents the development of an autonomous mobile robot based on ROS 2, capable of navigating a predefined track using visual line following and responding to environmental cues such as traffic signs and traffic lights. The system was designed using a modular architecture composed of multiple Python and C++ nodes, including components for image processing, state management, and auditory feedback. A dual PID control strategy was implemented to adjust the robot's angular velocity depending on the curvature of the path, ensuring smooth and accurate motion. Traffic perception modules were supported by YOLO-trained models, with a confidence-based filtering mechanism to increase detection reliability. The final system demonstrated robust performance in controlled environments, showing potential for adaptation in educational robotics, urban delivery platforms, or semi-structured navigation tasks. The results validate the effectiveness of the proposed architecture in achieving autonomous behavior under real-world constraints.

**Index Terms**—Autonomous navigation, buzzer feedback, computer vision, differential mobile robot, finite state machine, jerson nano, PID control, ROS2, traffic light, recognition, traffic light detection.

## I. INTRODUCTION

This project is developed within the context of a mobile robotics challenge, whose main objective is to achieve the autonomous navigation of a differential-drive robot along a track specifically designed for navigation testing. The track features curves, an intersection regulated by a traffic light, and various road signs such as ahead only, turn right, turn left, road work, stop, give way, representing a semi-structured environment that tests the robot's perception, decision-making, and control capabilities.

The development team received a kit from Manchester

Robotics which includes the structure and assembly instructions for the educational differential robot Puzzlebot, equipped with an NVIDIA Jetson Nano, a hackerboard for controlling DC motors with encoders, and a Raspberry Pi camera with a fish-eye lens.

The challenge focuses on implementing an autonomous system based on ROS 2 (Robot Operating System 2), enabling the robot to follow the black centerline of the track, accurately detect traffic signs along the route, and autonomously respond to traffic light signals at intersections.

This project provides an opportunity to integrate knowledge in computer vision, real time control, trajectory planning, and intelligent robotics.

## II. METHODOLOGY

The strategy was based on the creation of six ROS nodes programmed in Python: `state_machine`, `line_follower`, `traffic_light`, `traffic_signs`, `buzzer` and `camera`. The system was designed in a modular fashion to simplify debugging and facilitate the detection of individual errors. Only the `state_machine` node functions as a master node, orchestrating the pipeline and overall behavior of the entire system.

### A. Timeline of Activities and Responsibilities

The development began with the implementation of the line follower node, led by Brenda Cruz and Harum Kim. Initial testing was carried out on a track with a continuous line (no intersections). During this phase, a key improvement opportunity was identified: the onboard camera introduced significant latency. To address this, a dedicated camera node was implemented, which compresses the image before publishing it for use by other nodes. This was the only node developed in C++, as the required `image_transport` library was not available in Python. This change significantly improved the frame acquisition rate and overall real-time performance of the system.

For the traffic sign and traffic light detection nodes, a preparatory phase was carried out before implementation. YOLO models were trained using approximately 7,000 images for traffic signs (from a public dataset) and 1,200 images for traffic lights (captured directly on the test track). After training, the detection logic in Python was configured to only respond to detections with a confidence score greater than 0.70. The traffic sign detection node was developed by Mateo

---

<sup>1</sup>This work was supported as part of an academic project in the *Intelligent Robotics* course at Tecnológico de Monterrey, Campus Monterrey.

Ángelica N. Velázquez, Ximena L. Trejo, Brenda Cruz, Harum Kim, César M. Sánchez, Alondra Caspeta, and Valeria Meneses are undergraduate students at Tecnológico de Monterrey, Campus Monterrey, (e-mails: {A00838198, brenda.cruz, harum.kim, cesar.sanchez, alondra.caspeta, valeria.meneses}@tec.mx).

Sánchez and Valeria Meneses, while the traffic light detection node was handled by Alondra Caspeta.

The buzzer node was developed by Ximena Trejo, who tuned the frequency to emit specific tones corresponding to each traffic signal detected. Each action triggered by a traffic sign was assigned a unique sound for better feedback and debugging.

Finally, the integration of all nodes and the design of the decision-making logic for autonomous robot behavior were encapsulated in the `state_machine` node, developed by Nahomi Velázquez and Ximena Trejo. Two main states were defined for the robot to transition between along the route: `FOLLOW_LINE` and `INTERSECTION`. This node aggregates all relevant topics published by the other nodes and determines the robot's behavior based on the current environmental context and detected conditions.

### B. Block diagram

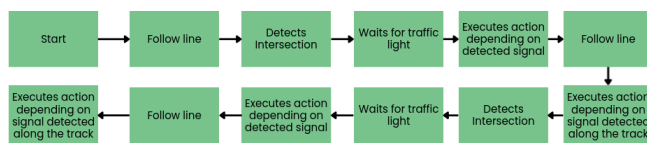


Figure 1. Block Diagram of the System.

### C. Control Strategy Implemented

To control the differential drive robot, two angular control strategies were implemented: a proportional (P) controller activated when the robot navigates through curves, and a proportional-integral (PI) controller, used during straight-line trajectories. The selection between these controllers is managed by a state machine, which switches between “curve” and “straight” states based on the error value that is defined as the difference between the center of the image and the detected line position.

Additionally, to prevent undesired accumulation of the integral term, when transitioning between states, all PID parameters are reset upon each state change. This ensures a stable and responsive control behavior across varying path geometries.

#### D. ROS Packages and Tools Used

Two primary ROS packages were used in this project: `custom_interfaces` and `final_te2003b`. The `custom_interfaces` package was created to define and implement custom service types required by the system. The `final_te2003b` package contains all the developed nodes mentioned throughout this report.

The following ROS tools and libraries were utilized to support system functionality:

- 1) `image_transport` (C++): Used to efficiently transmit images by enabling compression, which significantly reduces

latency and bandwidth usage.

- 2) **cv\_bridge**: Facilitates the conversion between ROS image messages and OpenCV image formats, enabling real-time image processing within the vision nodes.

### E. Algorithm Flowchart

- 1) line\_follower

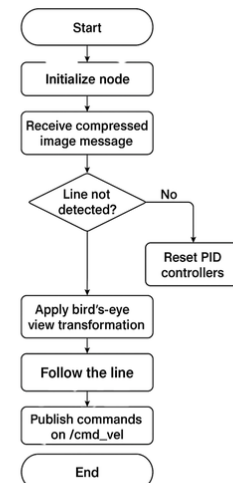


Figure 2. line follower flowchart.

- 2) camera

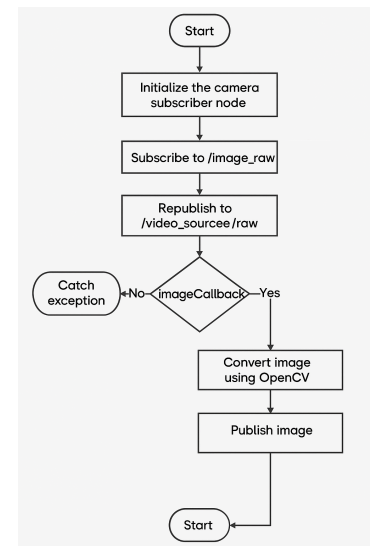


Figure 3, camera flowchart.

### 3) traffic\_signs

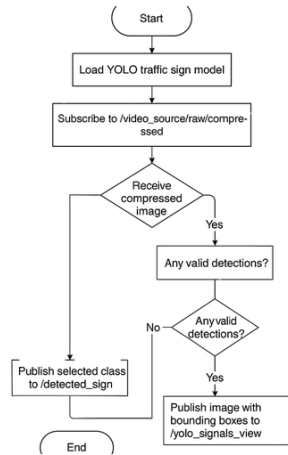


Figure 4. traffic\_signs flowchart.

### 4) traffic\_light

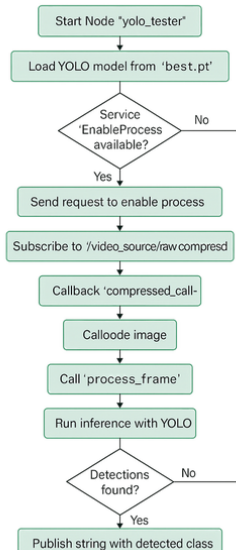


Figure 5. traffic\_light flowchart.

### 5) buzzer

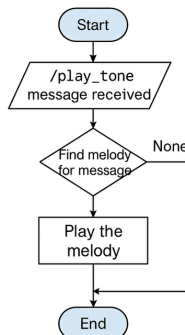


Figure 5. line\_buzzer flowchart.

### 6) state\_machine

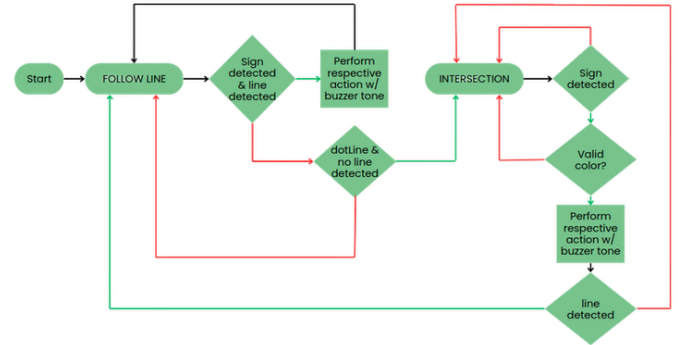


Figure 6. state\_machine flowchart.

### F. Node Communication

The system is composed of several ROS nodes that interact through the publication and subscription of defined topics. The communication structure is outlined below:

#### 1) camera

- **Subscribes to:** /image\_raw
- **Publishes to:** /video\_source/raw/compressed

This node converts raw images captured by the camera into a compressed format, optimizing image transmission and processing for the vision-based nodes.

#### 2) traffic\_signs

- **Subscribes to:** /video\_source/raw/compressed
- **Publishes to:** /detected\_sign

Detects traffic signs from the compressed image and sends the identified sign to the main control node.

#### 3) traffic\_light

- **Subscribes to:** /video\_source/raw/compressed
- **Publishes to:** /detected\_color

Identifies the current traffic light color. This information is only considered when the robot is in the INTERSECTION state.

#### 4) line\_follower

- **Subscribes to:** /velocity\_factor
- **Publishes to:** /line\_detected, /cmd\_vel

Responsible for following the black line on the track. It reports whether the line is detected and generates velocity commands to move the robot accordingly.

#### 5) buzzer

- **Subscribes to:**

1. /line\_detected (to detect transitions, such as entering intersections when a dotLine is followed by not\_detected).
2. /detected\_sign (to trigger specific actions based on recognized sign).
3. /detected\_color (used only when in the INTERSECTION state).

- **Publishes to:**

1. /cmd\_vel (to control robot motion).
2. /play\_tone (to activate the buzzer with specific melody).
3. /velocity\_factor (to adjust the robot's speed based on detected signals).

6) buzzer

- **Subscribes to:** /play\_tone

The line\_follower, traffic\_light, traffic\_signs, and buzzer nodes all publish key data to the state\_machine, which acts as the master control node and orchestrates the robot's autonomous behavior based on the inputs received.

### G. ROS2 Topic Used

Throughout the development of this project, multiple ROS2 topics were employed to enable communication between the various nodes responsible for image processing, motion control, and environmental awareness. Below is a description of the main topics used:

1) /image\_raw: Published by the camera node, this topic contains the raw image captured by the vision sensor.

2) /video\_source/raw/compressed: This compressed image stream is generated from /image\_raw using the image\_transport node. It is used by the traffic sign and traffic light detection nodes to reduce bandwidth consumption and processing latency.

3) /line\_detected: Published by the line\_follower node, this topic indicates whether the line on the track has been detected. The possible values are detected or not\_detected.

4) /velocity\_factor: Published by the state\_machine node, this topic defines a multiplicative factor applied to the robot's linear velocity. It is used to reduce speed in response to traffic signs such as road work or give way. The default value is 1.0.

5) /play\_tone: Output topic directed to the buzzer node. It carries specific melody identifiers to be played as an audible response to detected traffic signals.

6) /detected\_color: Published by the traffic\_light node, this topic communicates the currently detected traffic light color.

7) /detected\_sign: Published by the traffic\_signs node, this topic indicates which traffic sign has been identified in the input image.

8) /cmd\_vel: Published by both the line\_follower and state\_machine nodes, this topic sends linear and angular velocity commands to control the robot's movement.

### H. Data Communication Flow

The diagram illustrates the data communication flow between the various ROS2 nodes that comprise the autonomous robot system. The camera node captures raw image data and publishes it as a compressed stream, which is then processed by the traffic\_signs and traffic\_light nodes to detect road signs and traffic light colors, respectively. The detection results are published to the topics /detected\_sign and /detected\_color.

The line\_follower node subscribes to /velocity\_factor to dynamically adjust the robot's speed, and publishes /line\_detected and /cmd\_vel based on real-time line tracking.

The state\_machine node serves as the central decision-making component. It subscribes to /line\_detected, /detected\_sign, and /detected\_color to determine the robot's operational state. Based on these inputs, it issues movement commands via /cmd\_vel, triggers auditory feedback through /play\_tone, and modifies the robot's speed by publishing to /velocity\_factor.

Finally, the buzzer node listens to /play\_tone and plays specific audio cues in response, thereby completing the cycle of perception, decision-making, and actuation within the system.

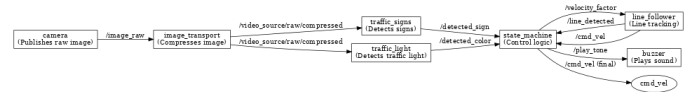


Figure 7. Communication Flow.

## III. MATERIALS

### A. Equipment Used

The execution of all ROS2 nodes, except for the buzzer node, was carried out on an ASUS ROG Strix G16 laptop equipped with a 13th-generation Intel Core i7-13650HX processor and an NVIDIA GeForce RTX 4060 Laptop GPU. The buzzer node was deployed on an NVIDIA Jetson Nano, allowing direct control of GPIO pins required for sound actuation.

### B. Hardware Connections

- 1) NVIDIA Jetson Nano: A compact yet powerful computer designed for embedded AI applications. It served as the controller for hardware-level interactions, such as the buzzer.

- 2) Hackerboard: Acts as the main control board of the Puzzlebot. It manages motor control and provides communication between sensors, actuators, and external computing devices (e.g., laptops, phones, Jetson Nano).
- 3) Power Bank: A 10,000 mAh power bank supporting PD 18W / QC 18W fast charging, with 1 USB-C output and 2 USB-A outputs, used to power both the Jetson Nano and the Hackerboard.
- 4) DC Brushless Fan: An axial cooling fan powered at 5V DC, used to prevent overheating during prolonged operation.
- 5) Passive Buzzer: A component capable of generating tones at different frequencies based on control signals received via GPIO.

### C. Hardware Connections

- 1) NVIDIA Jetson Nano:
  - Powered via USB-C input connected to the power bank
  - Connected to the Hackerboard through USB
 The passive buzzer is connected to GPIO pin 32, 5V, and GND
- 2) Hackerboard:
  - Powered by another power bank
  - Connected to the DC brushless fan through its 5V and GND ports for active cooling

## IV. RESULTS

### A. Demonstration Video

A video demonstrating the final behavior of the robot on the Puzzletrack was recorded and included as part of the project submission.

<https://youtu.be/AYWiGJdpJkY>

### B. Controller Gains

As mentioned previously, the line-following behavior was controlled using two distinct angular PID controllers depending on the type of trajectory. For curves, a proportional (P) controller was used with gains  $K_p=0.8$ ,  $K_i=0.0$ , and  $K_d=0.0$ , prioritizing a fast proportional response without integral accumulation or derivative influence. This configuration was sufficient for sharp directional changes where proportional correction alone proved effective.

For straight paths, the same proportional gain  $K_p=0.8$  was used, but an integral gain of  $K_i=0.13$  was introduced to eliminate small persistent

errors and keep the robot steadily centered on the line. In both cases, the derivative gain  $K_d$  was kept at zero, as it did not offer performance improvements during experiments and tended to amplify visual noise.

### C. Robot Behavior with Traffic Signs and Traffic Lights

The robot responded reliably to traffic signs due to the implementation of a confidence threshold. Only detections with a confidence score above 70% were considered valid, preventing false positives and improving robustness in real-world scenarios. This significantly enhanced the system's decision-making under variable lighting and detection conditions.

### D. Performance Analysis, Errors, and Improvement Opportunities

One of the main performance limitations was mechanical: the motors provided with the kit had defects that affected the Puzzlebot's mobility, causing delays in development and testing. In terms of perception, the initial YOLO model for traffic signs confused "turn left" and "turn right" signals. This issue was resolved by expanding the dataset with more representative images and retraining the model.

A key area for future improvement is the integration of odometry to execute turns based on positional feedback instead of relying on timers. Although development for this feature had started, it could not be completed due to hardware issues with the wheel encoders. Incorporating reliable odometry in future iterations would greatly enhance the precision of maneuvering, especially at intersections.

## V. CONCLUSIONS

The results obtained with the autonomous Puzzlebot demonstrate high reliability, achieved through the implementation of well-tuned PID controllers, a robust signal validation logic based on confidence thresholds, and an effective traffic light detection model that maintains accuracy even under varying lighting conditions. This system proves to be highly applicable in controlled or semi-structured environments, such as marked roadways equipped with clear signage and functional traffic lights.

Looking ahead, this approach can be scaled and adapted to more complex mobile platforms, applied in urban navigation systems for delivery robots, or integrated into educational autonomous vehicle projects—provided that similar visibility and environmental conditions are maintained.

### A. Individual Conclusions

1. Nahomi Velazquez. Participating in this project was a highly enriching experience that challenged me to apply and consolidate various areas of knowledge acquired during my academic formation. Working on an autonomous navigation system pushed me to think critically about real-world constraints and system

behavior under uncertainty. It also taught me the value of designing modular and scalable architectures for robotic applications. I take pride in having contributed to a functional and reliable system that integrates visual perception, control algorithms, and ROS-based communication. Beyond the technical achievements, this experience enhanced my problem-solving abilities, teamwork, and adaptability—skills that will undoubtedly be essential in my future as an engineer.

2. Harum Kim. This project allowed me to integrate concepts learned throughout my engineering specialization, putting into practice advanced topics such as computer vision, PID control, and object detection algorithms like YOLO. I learned the importance of proper sensor calibration and the need to anticipate the various scenarios a robot might encounter, as well as the necessity for a dynamic response to its environment.
3. Some of the technical skills I strengthened include real-time image processing, PID controller tuning, hardware-software integration, programming in Python and C++, and working with ROS. I also developed competencies in code debugging, sensor data analysis, and field testing with autonomous systems.
4. Mateo Sánchez. Working with Puzzlebot has been a journey combining logic and creativity. Seeing how a PID loop adjusts the speed of each wheel reminded me of the importance of precision in robotics, while the vision-based line follower taught me to rely on perception. Implementing YOLO to detect parts impressed me: the AI brings a level of adaptability reserved for science fiction. Assembling everything on top of ROS 2 showed the power of inter-node collaboration and modularity, facilitating iterations. But beyond the technical, reflecting on ethics made me aware of the impact of our creations: ensuring safety, avoiding bias and designing responsibly is as vital as any algorithm. Puzzlebot is not just a robot; it is a mirror of our way of tackling problems: with rigor, imagination and ethical commitment that drives robotics towards a more humane and sustainable future.
5. Valeria Meneses. Throughout this project, implementing a real system that integrates computer vision, real-time control, decision logic and communication in ROS2 has represented a big leap in my training. At first, many concepts such as image handling in ROS or PID control applied to motion were still a bit confusing to me despite having worked with these before. But seeing them working on a real robot and with all the problems that arose and that we had to solve, I was able to gain more experience and practice in these topics. I learned to not only program nodes, but to design a robust,

modular and functional system that responds to its environment autonomously.

In addition, working with YOLO models and training them from scratch helped me to better understand the entire process of machine learning, from data collection to implementation in a robot.

This project not only consolidated my technical skills, but also reinforced my confidence as a future engineer, proving that I am capable of creating complex solutions in a team and in a professional manner.

6. Alondra Caspeta. This project was a key learning experience that allowed me to apply advanced robotics concepts in a practical and structured way. I faced challenges that required critical thinking, such as ensuring reliable sensor readings, maintaining stability in control systems, and coordinating multiple ROS 2 nodes efficiently. Through this process, I strengthened my technical foundation in perception, control, and system integration, while also developing greater confidence in working with real-world robotics tools and environments. This experience has inspired me to continue growing in the field of autonomous systems and pursue new opportunities in intelligent robotics.

#### REFERENCES

- [1] [1] Ultralytics, "Ultralytics YOLOv8 Documentation," *Ultralytics*, 2024. [Online]. Available: <https://docs.ultralytics.com/>. [Accessed: Jun. 13, 2025].
- [2] [2] J. Lambert, "Train YOLOv8 on a Custom Dataset," *Roboflow Blog*, Apr. 13, 2023. [Online]. Available: <https://blog.roboflow.com/train-yolov8-custom-data/>. [Accessed: Jun. 13, 2025].